

教AI的陶老师作业训练营第四期（4）

如果在上次的作业中，你使用PyTorch实现了LeNet网络的定义的话，你应当已经知道一个网络的定义过程了。把对应的功能封装成函数后根据你对网络的设计将这些零件拼装在一起，现在我们会用到全连接层、卷积层、池化层和激活函数，之后随着网络设计的复杂，你会见到更多的零件，比如dropout、Batch normalization等等。所以先学理论，搞清楚这些零件的作用，以及网络的结构，用代码实现只是分分钟的事儿。

以下是一个可以用来实现LeNet网络的class，目的是之后实例出这个网络去跑MNIST数据集。上次有同学说定义网络不可思议，你看看是不是我说的零件拼装，搞定这个以后理论上你可以尝试自己去构建你的网络了。

In []:

```
import torch
import torch.nn as nn
class LeNet(nn.Module):
    def __init__(self):
        super(LeNet, self).__init__()
        self.conv1 = nn.Conv2d(1, 6, 5)
        self.pool1 = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.pool2 = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(16 * 4 * 4, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool1(torch.sigmoid(self.conv1(x)))
        x = self.pool2(torch.sigmoid(self.conv2(x)))
        x = x.view(-1, 16 * 4 * 4)
        x = torch.sigmoid(self.fc1(x))
        x = torch.sigmoid(self.fc2(x))
        x = self.fc3(x)
        return x
```

参考上例回答：

- ① __init__(self) 部分定义了什么？
- ② forward(self, x) 部分定义了什么？
- ③ nn.Linear 的作用是什么？
- ④ x.view(-1, 16 * 4 * 4) 的作用是什么？为什么需要这么做？
- ⑤ 怎么是"16 * 4 * 4"?论文的结构图中不是"16 * 5 * 5"?什么原因？

这段代码里面涉及到了Python类的定义和初始化方法等基础知识，不了解的话去看Python的函数

(<https://docs.python.org/zh-cn/3/tutorial/controlflow.html#defining-functions> (<https://docs.python.org/zh-cn/3/tutorial/controlflow.html#defining-functions>)) 与类 (<https://docs.python.org/zh-cn/3/tutorial/classes.html> (<https://docs.python.org/zh-cn/3/tutorial/classes.html>)) 这一部分。

模型被定义好后，使用之前需要先实例出来，比如：

In []:

```
net = LeNet()
```

下面来看看一个网络的训练是怎么进行的。
先来准备好数据，现在以MNIST数据集为例。

In []:

```
import numpy as np
train_X, test_X, train_y, test_y = np.load(
    './mnist.npy', allow_pickle=True)

x_train = train_X.reshape(60000, 28*28).astype(np.float32)
x_test = test_X.reshape(10000, 28*28).astype(np.float32)
```

现在我们要把数据集处理一下再用作训练，对于当前数据集，需要有以下几步：

1. 转换成 tensor 类型
2. 再转换成 Dataset 类型
3. 再生成 DataLoader

这几步是先后依赖的。我们想用大批量随机下降的方法来进行模型的训练，所以需要借助 DataLoader 来进行数据的批量加载-> DataLoader 在创建的时候需要数据是 Dataset 类型-> Dataset 类型的创建需要数据是 tensor 类型。

下面具体来说，首先将 NumPy 加载进来的数据类型 ndarray 转换成 tensor 类型：

In []:

```
x_train, y_train, x_test, y_test = map(
    torch.tensor, (x_train, train_y, x_test, test_y))
```

⑥ map 函数Python中很常使用到，在这里的作用是什么？

下面是生成 Dataset 类。

In []:

```
from torch.utils.data import TensorDataset
```

x_train 和 y_train 都可以合并为一个 TensorDataset，这将更易于迭代和切片。

In []:

```
train_ds = TensorDataset(x_train, y_train)
```

在我们自己定义数据集的时候，可以使用 PyTorch 的 TensorDataset 来实现这一步，可是⑦我们为什么需要这么做？

除此之外，Pytorch 的 DataLoader 负责批次管理。需要从 Dataset 创建一个 DataLoader。DataLoader 使迭代变得更加容易，可以自动为我们提供随机打乱的每个小批量。

In []:

```
from torch.utils.data import DataLoader
bs = 64
train_ds = TensorDataset(x_train, y_train)
train_dl = DataLoader(train_ds, batch_size=bs, shuffle=True)
```

现在，训练时使用的数据加载器已经处理好了，来写一个训练过程。

补充：

1. 通常管所有数据扫完一遍叫做一个epoch。

In []:

```
epochs = 32
```

2. Pytorch 还提供了一个包含各种优化算法的软件包 `torch.optim`。我们可以使用优化器中的`step`方法来进行参数的更新（注意：`loss.backward()`只是求得了每一个参数的梯度并且记录在参数对应的节点上，但是参数并不更新）。

In []:

```
from torch import optim
opt = optim.SGD(net.parameters(), lr=0.01) # 定义一个优化器
```

3. 学习率记作 `lr`

In []:

```
lr = 0.01
```

4. 损失函数也提前准备好

In []:

```
import torch.nn.functional as F
loss_func = F.cross_entropy
```

现在开始训练吧。

In []:

```

for epoch in range(epochs):
    for xb, yb in train_dl:
        xb = torch.reshape(xb, [-1, 1, 28, 28])
        pred = net(xb)
        loss = loss_func(pred, yb)
        loss.backward()
        opt.step()
        opt.zero_grad()
    print(epoch, loss)

```

上例中，⑧ `torch.reshape(xb, [-1, 1, 28, 28])` 的作用是什么？

⑨ `opt.zero_grad()` 的作用是什么？为什么需要有这一步？

⑩ 尝试对 `bs` 和 `epochs` 进行调整再观察训练过程。

LeNet 就说到这里，下面来看另一个经典网络 "AlexNet"。相对于 LeNet，AlexNet 做的改进有：

1. 更深的网络结构。AlexNet 使用了比 LeNet 更深的网络结构，包含 5 层卷积层和 3 层全连接层，这样可以提取更高级别的特征，进而提高模型的准确率。
2. ReLU 激活函数的使用。AlexNet 在每个卷积层之后都使用了 ReLU 激活函数，这个激活函数相比于 sigmoid 激活函数可以大大减少梯度消失的问题，进而训练更深的神经网络。
3. Dropout。AlexNet 使用 Dropout 技术在全连接层中加入随机丢弃部分神经元的操作，来防止过拟合，提高模型的泛化能力。

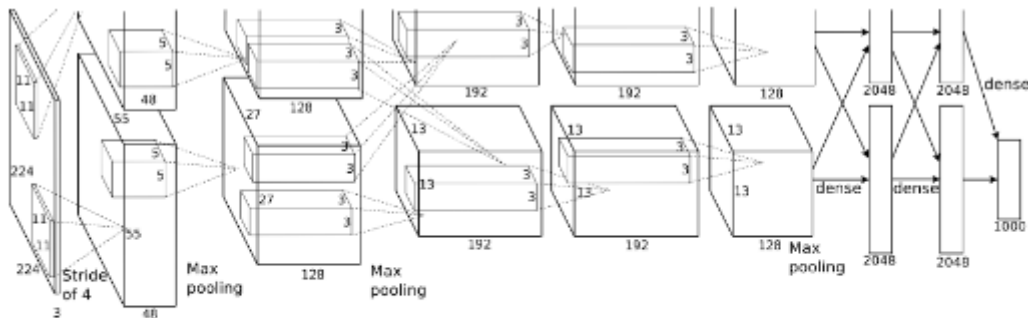


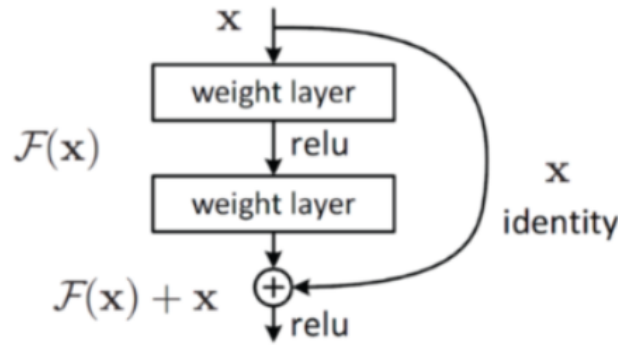
Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

⑪ 用到的新零件有哪些？如何定义？

⑫ 请尝试定义 AlexNet 网络，然后将上述过程对应实现

⑬ 在同样的数据集、`bs` 和 `lr` 等超参数设置下，两个网络的训练效果差别是怎样的？

下面进行下一个网络，何恺明大神的 "ResNet"。



这绝对称得上是"中国之光", 这个网络的出现, 让深度学习网络模型真正可以变深了。请参考我的视频"ResNet 残差神经网络"回答下列问题:

⑭请画出通道数有变化和无变化两种情况下的残差块结构。

⑮在无通道数变化情况下, 进入残差块的数据是 x , 那么输出可以表示为什么? 中间的计算过程是怎样的?

⑯引入残差块解决的是什么问题? 请用公式推倒证明。

⑰对于ResNet你还有哪些认识? (背景、作者、影响、延伸等方向不限)

如果你有信心, 也可以自己去定义这个网络, 这里就不作要求了。但如果你想使用这个网络, 可以在PyTorch中直接下载调用预训练好的模型。

⑱如何下载调用? 请下载后重新对MNIST数据集进行训练。

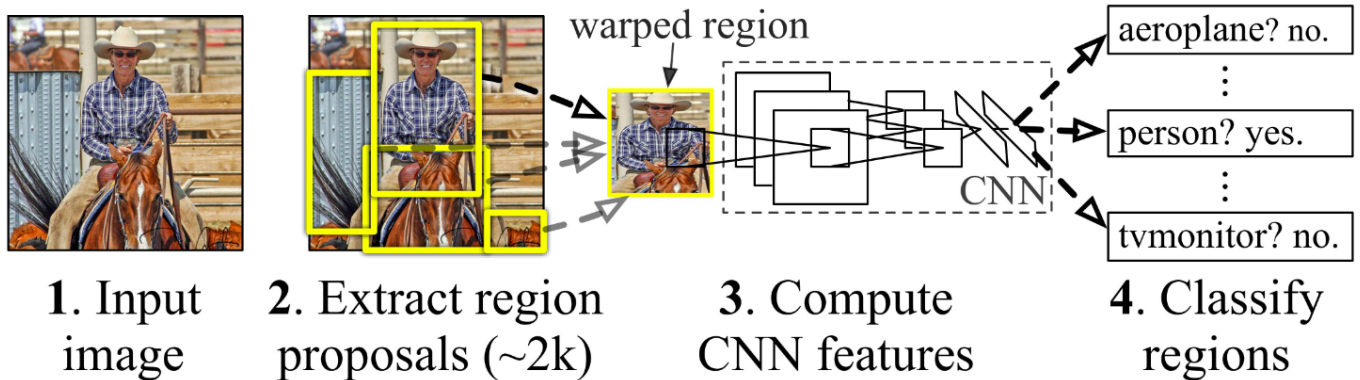
上面的模型大多用来对图片进行分类, 下面来说图像处理中的另一个主要任务: 目标检测。

⑲相对于图片分类来说, 目标检测是怎样的任务?

目标检测中有两种常见解决思路, two stages 和 one stage。

其中R-CNN网络可以看做是 two stages 的代表, 后续还有Fast-RCNN和Faster-RCNN。

R-CNN: *Regions with CNN features*



看我的视频"R-CNN的设计思想"后, ⑳简述"R-CNN的设计思想"

one stage 的代表当然就是你一定听过的YOLO模型了, 我们来以YOLOv1为例进行学习。

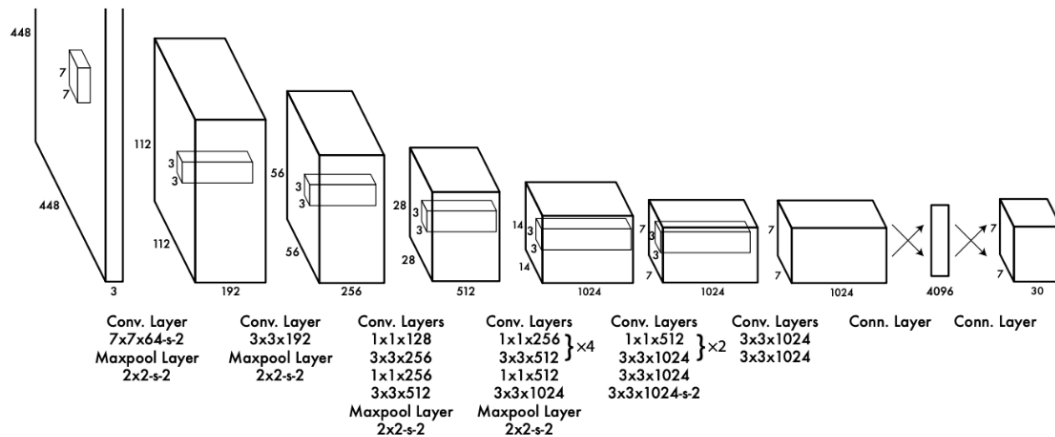
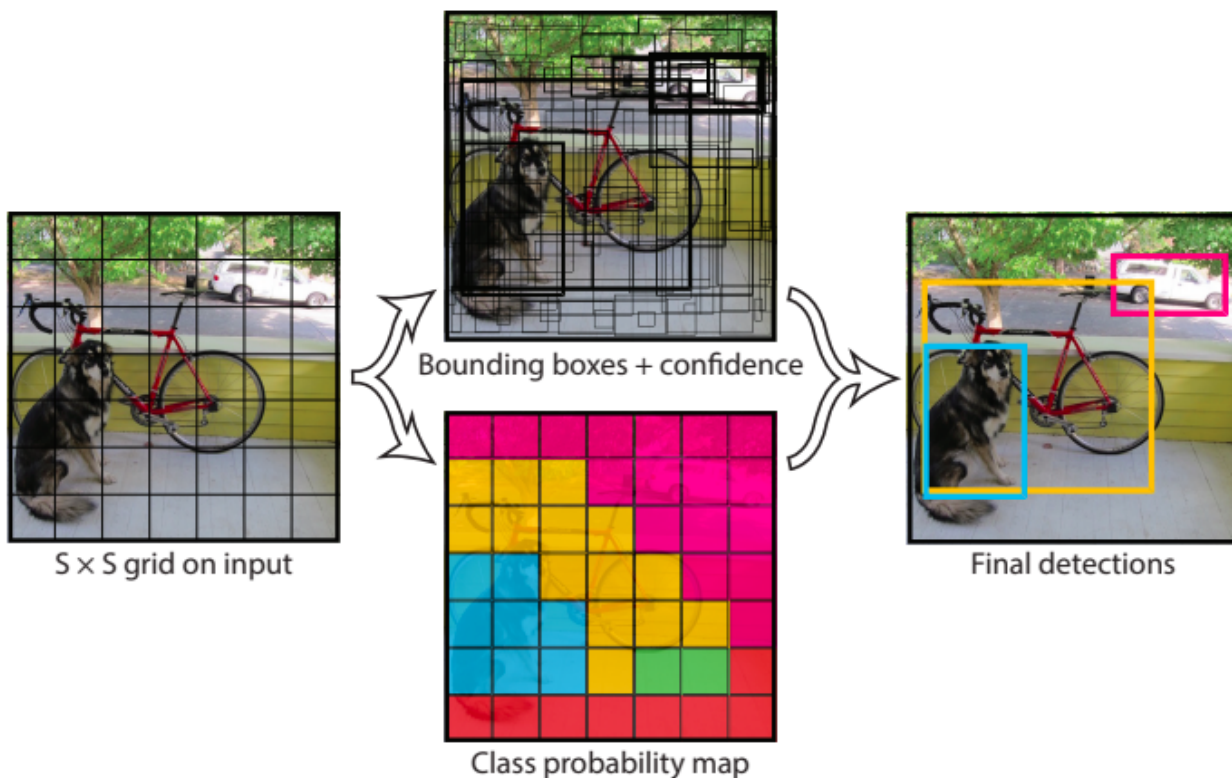


Figure 3: The Architecture. Our detection network has 24 convolutional layers followed by 2 fully connected layers. Alternating 1×1 convolutional layers reduce the features space from preceding layers. We pretrain the convolutional layers on the ImageNet classification task at half the resolution (224×224 input image) and then double the resolution for detection.

①测试过程中，网络的输入图像是多大？

②输出的是 $7 \times 7 \times 30$ 的张量，每一个位置的数值分别代表什么意义？

③参考下图解释，模型预测出来的框是98个，之后经过哪些处理才变为下图中的3个框？



④训练过程中，选用的损失函数是怎样的？请详细解释。

造不造轮子这个看个人，但是使用轮子的话，把具体内容了解清楚了也肯定会收益。强烈建议有余力的同学，将本次内容中涉及到的所有论文进行精读，都非常经典。

